Q1) Create two tables, influencers and brands, with at least 3 sample rows each. Use a FULL OUTER JOIN to list all influencers and brands, showing influencer_name and brand_name, matching on city. If there is no match, display NULL for the missing side.

Hint: Use LEFT JOIN, RIGHT JOIN, and UNION if your SQL dialect does not support FULL OUTER JOIN directly.

```
SELECT
    i.influencer_name,
    b.brand_name
FROM influencers i
LEFT JOIN brands b
    ON i.city = b.city

UNION

SELECT
    i.influencer_name,
    b.brand_name
FROM influencers i
RIGHT JOIN brands b
    ON i.city = b.city;
```

Q2) Given a table called playlists with columns (id, playlist_name, parent_playlist_id), write a SELF JOIN query to display each playlist alongside its parent playlist's name, similar to how Spotify might nest playlists.

```
SELECT
    p.playlist_name AS
playlist_name,
    parent.playlist_name
AS parent_playlist_name
FROM playlists p
LEFT JOIN playlists
parent
    ON
p.parent_playlist_id =
parent.id;
```

Q3) Create two tables: users and offers. Write a CROSS JOIN query to generate all possible combinations of users and offers, displaying user_name and offer_title. Explain in a comment how this could be used for a Flipkart-style personalized offer campaign.

```sql
SELECT
    u.user_name,
    o.offer_title
FROM users u
CROSS JOIN offers o;

-- Flipkart-style use:
-- Generate every
possible user-offer
combination to prepare
-- a campaign. Later,
filter offers based on
user preferences,
-- purchase history, or
eligibility before
sending them.
```

Q4) You have an employees table with columns (id, name, manager_id). Write a SELF JOIN to display each employee's name along with their manager's name. Then, modify your query to only show employees who do not have a manager (i.e., top-level managers).

```sql
SELECT
    e.name AS
employee_name
FROM employees e
LEFT JOIN employees m
    ON e.manager_id =
m.id
WHERE e.manager_id IS
NULL;
```

Q5) Use ChatGPT or Copilot to help you write a SQL query that finds all pairs of users from a users table who live in the same city (excluding pairs where the user is compared with themselves). Paste the query and briefly describe how the AI helped you improve or debug it.

```sql
SELECT
    u1.username AS user1,
    u2.username AS user2,
    u1.city
FROM users u1
INNER JOIN users u2
    ON u1.city = u2.city
    AND u1.user_id <
u2.user_id;
```